

02 — AI GAP ANALYSIS (Phase 4)

- **Sinh viên:** Ninh Văn Khải — MSSV **23127060**
- **Đối tượng review:** 3 spec + 5 Page Object do AI sinh ở Phase 3 (80 test), cập nhật thêm GAP-08/09 phát hiện ở Phase 8 (83 test)
- **Cách làm:** chạy thật → đọc lại từng assertion → soi anti-pattern §11 SKILL → sửa → chạy lại

Nguyên tắc: chỉ ghi vấn đề **có bằng chứng** (log lỗi thật, hoặc dòng code cụ thể).

Không liệt kê rủi ro lý thuyết để làm dày báo cáo.

1. Kết quả quét anti-pattern tự động

```
grep -rn "waitForTimeout" tests/           ⇒ chỉ khớp trong COMMENT, 0 lời gọi thật
grep -rn "locator('\.|locator(\"\".\" tests/ ⇒ 0 (không dùng class Tailwind)
grep -rn "nth-child|xpath=" tests/         ⇒ 0
grep -rn "^const cases = \[" tests/*.spec.js ⇒ 0 (không hardcode data inline)
npx playwright test --list --project=chromium ⇒ Total: 80 tests in 3 files
```

2. Bảng Gap Analysis

#	Vấn đề	Loại	AI sai vì sao	Cách sửa
GAP-00	<code>getByRole('textbox')</code> ở bước 2 của <code>/forgot-password</code> khớp 2 phần tử (ô OTP + ô mật khẩu) → <code>strict mode violation</code> , 9/30 test FR-03 fail	fragile selector	Model limitation. AI khẳng định chắc nịch trong Page Object rằng "Playwright không expose role cho <code>input[type=password]</code> ". Đây là kiến thức API sai — Playwright có gán role <code>textbox</code> cho input password. AI viết comment giải thích cho một giả định chưa hề kiểm chứng.	Đổi sang <code>.first()</code> (ô OTP đứng trước trong DOM), giữ <code>input[type="password"]</code> cho ô mật khẩu và ghi lại nguyên nhân trong comment
GAP-01	<code>expect(c.expect.minRows).toBe(SEED_PRODUCT_COUNT)</code> — so hằng số với hằng số, không chạm tới SUT	weak assertion	Prompt quality. Yêu cầu "≥3 assertion pattern mỗi test" khiến AI nhồi thêm assertion cho đủ số lượng thay vì đủ ý nghĩa. Đây đúng là anti-pattern <code>expect(true).toBe(true)</code> trá hình.	Giữ dòng đó nhưng hạ xuống vai trò "kiểm tra data file khớp seed", thêm assertion thật: số dòng trên UI phải bằng <code>GET /api/products.length</code>
GAP-02	FR15-TC05 đếm <code>totalRows</code> từ rất sớm rồi mới thao tác sửa — giữa hai mốc đó có nhiều bước, worker song song có thể thêm/xoá sản phẩm	flaky wait	Đặc thù feature. Test chạy <code>fullyParallel</code> trên một CSDL SQLite dùng chung. AI viết test như thể mình đọc chiếm dữ liệu.	Dời phép đếm xuống ngay trước khi bấm Lưu, và chờ dòng mục tiêu xuất hiện trước đó
GAP-03	<code>expect(xssExecuted).toBe(!c.expect.scriptDidNotExecute)</code> — phủ định kép	weak assertion	Model limitation. AI cố ép mọi giá trị kỳ vọng phải đọc từ data file, kể cả khi việc đó làm assertion không đọc nổi. Người review rất dễ hiểu ngược.	Viết thẳng <code>.toBe(false)</code> cho hành vi, tách riêng 1 dòng đối chiếu data file
GAP-04	<code>openProductsTab()</code> chỉ chờ nút "Lưu sản phẩm" — nút đó thuộc form , có ngay lập tức, trong khi <code>fetchData()</code> vẫn đang chạy → test đếm dòng có thể đọc bảng rỗng	flaky wait	Đặc thù feature. SPA render form và bảng độc lập nhau; AI chọn phần tử "thấy được sớm nhất" làm mốc chờ mà không phân biệt phần tử nào phụ thuộc dữ liệu.	Chờ thêm nút <code>Sửa</code> đầu tiên (chỉ tồn tại khi đã có dữ liệu dòng)
GAP-05	FR08-TC01 chỉ kiểm tra "giỏ có tiền > 0", không hề kiểm tra đúng sản phẩm nào đã vào giỏ	missing edge case	Prompt quality. Case trong data file mô tả "happy path checkout" nên AI tối ưu cho việc đi hết luồng, bỏ qua tính đúng đắn của dữ liệu giữa chừng.	Ghi lại tên sản phẩm ở Home rồi assert dòng đó có mặt trong bảng giỏ hàng; thêm <code>rowByProductName()</code> vào <code>CartPage</code>
GAP-06	Biến thể token 9999 trong CSV có xác suất 1/9000 trùng token thật → test kỳ vọng 400 sẽ ngẫu nhiên nhận 200	flaky wait	Đặc thù feature. Token của SUT chỉ 4 chữ số nên không gian trùng rất nhỏ. AI ban đầu coi "token bịa ra" là chắc chắn sai.	Xin lại token cho tới khi khác giá trị biến thể, kèm comment nêu rõ đây là chống trùng chứ không phải retry che lỗi
GAP-07	Trình duyệt <code>firefox / webkit</code> chưa được tải → toàn bộ test UI fail sau 3 ms với <code>browserType.launch: Executable doesn't exist</code>	môi trường	Model limitation. AI mặc định môi trường đã sẵn sàng vì chromium chạy được. Kết quả "52 passed" trông như thành công một phần, thực chất là 28 test không chạy nổi.	<code>npx playwright install firefox webkit</code> trước khi vào Phase 5
GAP-08	<code>CartPage.itemRowCount()</code> gọi <code>isVisible()</code> ngay sau khi điều hướng — <code>isVisible()</code> không chờ , nên khi React chưa render xong thì hàm trả về <code>0 - 1 = -1</code>	flaky wait	Đặc thù feature + model limitation. AI dùng <code>isVisible()</code> như một phép kiểm tra tức thời mà quên rằng nó không có cơ chế chờ như <code>expect(...).toBeVisible()</code> . Trên chromium đủ nhanh nên không lộ; chỉ webkit mới làm fail .	Chờ trang chốt trạng thái bằng <code>expect(emptyMessage.or(cartHeading)).toBeVisible()</code> rồi mới đếm
GAP-09	FR15-TC05 đếm số dòng ngay sau <code>await dialogPromise</code> — nhưng đóng alert không đồng nghĩa React đã render lại bảng	flaky wait	Đặc thù feature. AI coi "alert đã đóng" là mốc an toàn để đọc DOM. Thực tế <code>setProducts()</code> chạy trước alert nhưng React render bất đồng bộ sau đó.	Chờ <code>expect(rowByName(newName).first()).toBeVisible()</code> rồi mới đếm

3. Bốn lỗi AI đáng chú ý nhất (dùng cho AI Critique)

- AI tự tin về API mình không nắm chắc (GAP-00).** Nguy hiểm nhất không phải cái sai, mà là AI **viết comment giải thích** cho cái sai đó — làm người đọc tin rằng nó đã được kiểm chứng. Chỉ có chạy thật mới lộ ra.
- Đếm số lượng assertion thay vì chất lượng (GAP-01, GAP-03).** Khi rubric nói "≥3 assertion pattern", AI có xu hướng thỏa mãn con số bằng những assertion không kiểm thử gì.
- Bỏ qua tính đồng thời và trạng thái môi trường (GAP-02, GAP-04, GAP-07).** AI viết test như thể chỉ có một mình nó chạy, trên một máy đã cài đặt hoàn hảo.
- Chi kiểm chứng trên trình duyệt nhanh nhất (GAP-08, GAP-09).** Hai lỗi chờ này **pass sạch trên chromium ở cả --repeat-each=2**, chỉ lộ ra khi chạy webkit ở Phase 8. Bài học: "ổn định 2 lần liên tiếp trên 1 browser" **không** đồng nghĩa với ổn định. Cả hai bắt nguồn từ cùng một thói quen — lấy phép đọc DOM

không có cơ chế chờ (`isVisible()` , `count()`) làm mốc, thay vì web-first assertion.

4. Kết quả sau khi sửa

```
npx playwright test --project=chromium
⇒ 80 passed (17.0s)      ← trước khi sửa: 80 passed nhưng có 5 điểm yếu ở bảng trên

# Sau khi bổ sung test ở Phase 8 và vá GAP-08 / GAP-09:
npx playwright test --project=webkit
⇒ 83 passed (35.8s)

npx playwright test --project=webkit --grep "FR08-TC17|FR15-TC05" --repeat-each=3   (chạy 3 lượt)
⇒ 6 passed · 6 passed · 6 passed   ← 18/18, hai test từng flaky nay ổn định

node scripts/run-multibrowser.mjs
⇒ 9/9 run PASS toàn bộ (249 test)
```

Chi tiết số liệu 9 lần chạy multi-browser: xem `report/03-RUN-SUMMARY.md` (Phase 5).

5. 🧑 Chờ Khả review & ký

- [] Đọc lại 9 GAP ở §2, xác nhận cách sửa hợp lý.
- [] Quyết định giữ / bỏ những assertion "đối chiếu data file" còn lại (GAP-01, GAP-03).
- [] Ký xác nhận "**đã review script**": _____